Numpy Module

Importing Numpy

In [ ]:
```
import numpy as np
```

Create array from a list

In [ ]:
```
arr = np.array([1,2,3,4])
print(arr)
print(type(arr))
```

```
[1 2 3 4]
<class 'numpy.ndarray'>
```

Dimension of an array

In [ ]:
```
print(arr.ndim)
```

```
1
```

Creating a 0-D array

In [ ]:
```
arr1 = np.array(67)
print(type(arr1))
```

```
<class 'numpy.ndarray'>
```

Creating a 1-D array

In [ ]:
```
arr = np.array([1,2,3,4])
print(arr)
print(type(arr))
```

```
[1 2 3 4]
<class 'numpy.ndarray'>
```

Create a 2-D Array

In [ ]:
```
arr2 = np.array([[1,2,3],[4,5,6]])
print('Dimension of Array arr2: ', arr2.ndim)
print(arr2)
```

```
Dimension of Array arr2:  2
[[1 2 3]
 [4 5 6]]
```

Create a 3-D Array

In [ ]:
```
arr3 = np.array([[[1,2,3],[4,5,6]],[[1,2,3],[4,5,6]]])
print("Dimension of arr3:",arr3.ndim)
print(arr3)
```

```
Dimension of arr3: 3
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

Creating N-D Array

In [ ]:
```
arr4 = np.array([1,2,3,4], ndmin=5)
print(arr4.ndim)
print(arr4)
```

```
5
[[[[[1 2 3 4]]]]]
```

Array Indexing

In [ ]:
```
arr = np.array([1,2,3,4])
print(arr[0])
print(arr[1])
```

```
1
2
```

In [ ]:
```
arr = np.array([[1,2,3,4],[5,6,7,8]])
print("Dimension of array: ",arr.ndim)
print(arr)
print(arr[0])
print(arr[0][3])
```

```
Dimension of array:  2
[[1 2 3 4]
 [5 6 7 8]]
[1 2 3 4]
4
```

Adding Elements of Arrays

In [ ]:
```
print(arr[0][3]+ arr[1][3])
print(arr[0]+arr[1])
```

```
12
[ 6  8 10 12]
```

Array Slicing

In [ ]:
```
arr = np.array([1,2,9,3,4,5,6,7,8])
print(arr[1:4])
```

```
[2 9 3]
```

In [ ]:
```
arr = np.array([[1,2,3,4],[5,6,7,8]])
print("Dimension of array: ",arr.ndim)
print(arr[0][:2])
print(arr[1][:2])
print(arr[1,:2])
print(arr[0:2,:2])
```

```
Dimension of array:  2
[1 2]
[5 6]
[5 6]
[[1 2]
 [5 6]]
```

Data Types in Numpy

We have python datatype with others as below:

- i - Integer
- b - Boolean
- u - Unsigned Integer
- f - Float
- c - Complex
- m - timedelta
- M - datetime
- O - Object
- S - String
- U - unicode string
- V - void

In [ ]:
```
arr = np.array([1,2,3,4])
print(arr.dtype)
```

```
int64
```

In [ ]:
```
arr = np.array([1,2,3,4],dtype='S')
print(arr)
print(arr.dtype)
```

```
[b'1' b'2' b'3' b'4']
|S1
```

In [ ]:
```
arr = np.array([1,2,3,4],dtype='i4')
print(arr)
print(arr.dtype)
```

```
[1 2 3 4]
int32
```

In [ ]:
```
arr = np.array([1,2,3,4],dtype='i')
print(arr)
print(arr.dtype)
```

```
[1 2 3 4]
int32
```

Conversion of datatype while creating

In [ ]:
```
arr = np.array(['1','2','3','4'],dtype='i')
print(arr)
print(arr.dtype)
```

```
[1 2 3 4]
int32
```

The 'a' is not able to convert hence, will give an error

In [ ]:
```
arr = np.array(['1','a','3','4'],dtype='i')
print(arr)
print(arr.dtype)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[133], line 1
----> 1 arr = np.array(['1','a','3','4'],dtype='i')
      2 print(arr)
      3 print(arr.dtype)

ValueError: invalid literal for int() with base 10: 'a'
```

In [ ]:
```
arr = np.array(['2.1','2.3','3.5','4.5'],dtype='f')
print(arr)
print(arr.dtype)
```

```
[2.1 2.3 3.5 4.5]
float32
```

Converting Data Type after conversion

In [ ]:
```
newarr = arr.astype('i')
print(newarr)
print(newarr.dtype)
```

```
[2 2 3 4]
int32
```

In [ ]:
```
newarr = arr.astype(int)
print(newarr)
print(newarr.dtype)
```

```
[2 2 3 4]
int64
```

In [ ]:
```
arr = np.array([1,0,3])
newarr = arr.astype(bool)
print(newarr)
print(newarr.dtype)
```

```
[ True False  True]
bool
```

Array Copy and View

When copy() method is used, the original and copied arrays remain seperate

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
newarr = arr.copy()
arr[2] = 78

print(arr)
print(newarr)
```

```
[ 1  2 78  4  5  6  7  8]
[1 2 3 4 5 6 7 8]
```

When view() method is used, both the arrays are affected

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
print(arr)
newarr = arr.view()
arr[2] = 78

print(arr)
print(newarr)
```

```
[1 2 3 4 5 6 7 8]
[ 1  2 78  4  5  6  7  8]
[ 1  2 78  4  5  6  7  8]
```

By using .base, we can check who owns the data

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
#print(arr)
newarr = arr.view()
arr[2] = 78

print(arr.base)
print(newarr.base)
```

```
None
[ 1  2 78  4  5  6  7  8]
```

Shape

We can use shape to see the shape of the array

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
print("shape of arr is:",arr.shape)
print("N-D of arr:",arr.ndim)
print(arr)
arr1 = np.array([[1,2,3,4],[5,6,7,8],[13,14,15,16]])
print("shape of arr1 is",arr1.shape)
print("N-D of arr:",arr1.ndim)
print(arr1)
arr3 = np.array([[[1,2,3],[4,5,6]],[[1,2,3],[4,5,6]],[[1,2,3],[4,5,6]]])
print("shape of arr1 is",arr3.shape)
print("N-D of arr:",arr3.ndim)
print(arr3)
```

```
shape of arr is: (8,)
N-D of arr: 1
[1 2 3 4 5 6 7 8]
shape of arr1 is (3, 4)
N-D of arr: 2
[[ 1  2  3  4]
 [ 5  6  7  8]
 [13 14 15 16]]
shape of arr1 is (3, 2, 3)
N-D of arr: 3
[[[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]

 [[1 2 3]
  [4 5 6]]]
```

Reshape

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
print("shape of arr1 is",arr.shape)
print("N-D of arr:",arr.ndim)
print(arr)

newarr = arr.reshape(4,2)
print("shape of newarr is",newarr.shape)
print("N-D of newarr:",newarr.ndim)
print(newarr)
```

```
shape of arr1 is (8,)
N-D of arr: 1
[1 2 3 4 5 6 7 8]
shape of newarr is (4, 2)
N-D of newarr: 2
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8,9,10,11,12])
print("shape of arr1 is",arr.shape)
print("N-D of arr:",arr.ndim)
print(arr)

newarr1 = arr.reshape(4,3)
print("shape of newarr1 is",newarr1.shape)
print("N-D of newarr1:",newarr1.ndim)
print(newarr1)

newarr2 = arr.reshape(2,3,2)
print("shape of newarr2 is",newarr2.shape)
print("N-D of newarr2:",newarr2.ndim)
print(newarr2)
```

```
shape of arr1 is (12,)
N-D of arr: 1
[ 1  2  3  4  5  6  7  8  9 10 11 12]
shape of newarr1 is (4, 3)
N-D of newarr1: 2
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
shape of newarr2 is (2, 3, 2)
N-D of newarr2: 3
[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
```

Concatenating two arrays

In [ ]:
```
arr1 = np.array([1,2,3])
arr2 = np.array([4,5,6])

arr = np.concatenate((arr1,arr2))
print(arr)
```

```
[1 2 3 4 5 6]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.concatenate((arr1,arr2))
print(arr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [11 12 13]
 [14 15 16]]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.concatenate((arr1,arr2), axis=1)
print(arr)
```

```
[[ 1  2  3 11 12 13]
 [ 4  5  6 14 15 16]]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.stack((arr1,arr2))
print(arr)
```

```
[[[ 1  2  3]
  [ 4  5  6]]

 [[11 12 13]
  [14 15 16]]]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.stack((arr1,arr2),axis=1)
print(arr)
```

```
[[[ 1  2  3]
  [11 12 13]]

 [[ 4  5  6]
  [14 15 16]]]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.hstack((arr1,arr2))
print(arr)
```

```
[[ 1  2  3 11 12 13]
 [ 4  5  6 14 15 16]]
```

In [ ]:
```
arr1 = np.array([[1,2,3],[4,5,6]])
arr2 = np.array([[11,12,13],[14,15,16]])
arr = np.vstack((arr1,arr2))
print(arr)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [11 12 13]
 [14 15 16]]
```

Splitting arrays

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
newarr = np.array_split(arr,4)
print(newarr)
print(type(newarr))
```

```
[array([1, 2]), array([3, 4]), array([5, 6]), array([7, 8])]
<class 'list'>
```

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7])
newarr = np.array_split(arr,4)
print(newarr)
print(type(newarr))
```

```
[array([1, 2]), array([3, 4]), array([5, 6]), array([7])]
<class 'list'>
```

In [ ]:
```
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
newarr = np.array_split(arr,2)
print(newarr)
print(type(newarr))
```

```
[array([[1, 2],
       [3, 4]]), array([[5, 6],
       [7, 8]])]
<class 'list'>
```

In [ ]:
```
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
newarr = np.array_split(arr,2, axis=1)
print(newarr)
print(type(newarr))
```

```
[array([[1],
       [3],
       [5],
       [7]]), array([[2],
       [4],
       [6],
       [8]])]
<class 'list'>
```

In [ ]:
```
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
newarr = np.vsplit(arr,2)
print(newarr)
print(type(newarr))
```

```
[array([[1, 2],
       [3, 4]]), array([[5, 6],
       [7, 8]])]
<class 'list'>
```

In [ ]:
```
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
newarr = np.hsplit(arr,2)
print(newarr)
print(type(newarr))
```

```
[array([[1],
       [3],
       [5],
       [7]]), array([[2],
       [4],
       [6],
       [8]])]
<class 'list'>
```

Search

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
n = np.where(arr == 7)
print(n)
```

```
(array([6]),)
```

In [ ]:
```
arr = np.array([1,2,3,2,5,2,7,8,2])
n = np.where(arr == 2)
print(n)
```

```
(array([1, 3, 5, 8]),)
```

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
n = np.where(arr%2 == 0)
print(n)
```

```
(array([1, 3, 5, 7]),)
```

In [ ]:
```
arr = np.array([[1,2],[3,4],[5,6],[7,8]])
n = np.where(arr == 7)
print(n)
print(arr[3,0])
print(arr[3][0])
```

```
(array([3]), array([0]))
7
7
```

In [ ]:
```
arr = np.array([1,2,3,4,5,6,7,8])
n = np.searchsorted(arr, 7)
print(n)
```

```
6
```

In [ ]:
```
arr = np.array([1,2,3,2,5,2,7,8,2])
n = np.searchsorted(arr, 2)
print(n)
```

```
1
```

In [ ]:
```
arr = np.array([1,2,3,2,5,2,7,8,2])
#n = np.searchsorted(arr, 6)
#print(n)
#n = np.searchsorted(arr, 9)
#print(n)
#print(arr)
n = np.searchsorted(arr, 5)
print(n)
n = np.searchsorted(arr, 4)
print(n)
```

```
4
4
```

In [ ]:
```
arr = np.array([6,7,8,9])
n = np.searchsorted(arr, 7, side='right')
print(n)
n = np.searchsorted(arr, 4, side='right')
print(n)
n = np.searchsorted(arr, 10, side='right')
print(n)
```

```
2
0
4
```

In [ ]:
```
arr = np.array([1,3,5,7])
n = np.searchsorted(arr, 7, side='right')
print(n)
n = np.searchsorted(arr, 6, side='right')
print(n)
```

```
4
3
```

Sort

In [ ]:
```
arr = np.array([1,2,3,2,5,2,7,8,2])
newarr = np.sort(arr)
print(arr)
print(newarr)
```

```
[1 2 3 2 5 2 7 8 2]
[1 2 2 2 2 3 5 7 8]
```

In [ ]:
```
arr = np.array([[2,4,3],[5,1,0]])
newarr = np.sort(arr)
print(arr)
print(newarr)
```

```
[[2 4 3]
 [5 1 0]]
[[2 3 4]
 [0 1 5]]
```

Filter

In [ ]:
```
arr = np.array([25,32,45,96])
x = [True,False,True,False]

newarr = arr[x]
print(newarr)
```

```
[25 45]
```

In [ ]:
```
arr = np.array([25,32,45,96])
newarr = arr[arr%2 == 0]
print(newarr)
```

```
[32 96]
```

Generating Distributions

In [ ]:
```
from numpy import random

x = random.normal(size=(2,3))
print(x)
```

```
[[ 0.38695286 -0.73483219 -0.45710796]
 [ 0.90098595 -1.71620487  1.24063279]]
```